

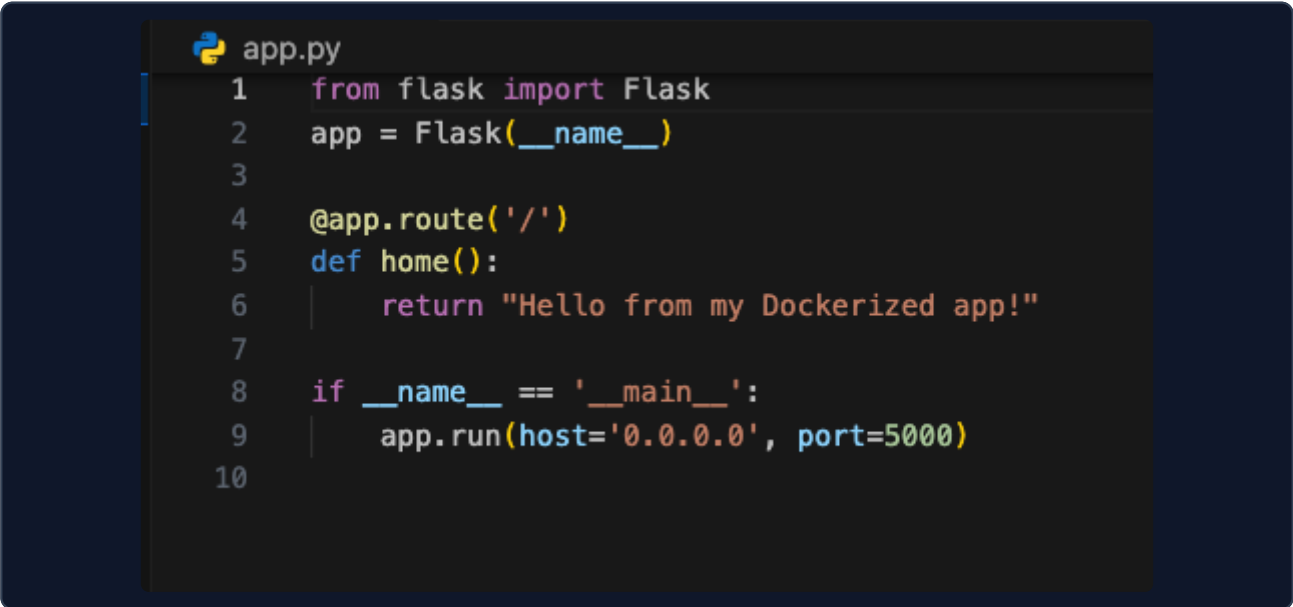
Containerization with Docker — Lab Task

RHOMBIXTECHNOLOGIES

Part A — Project Files

1. app.py

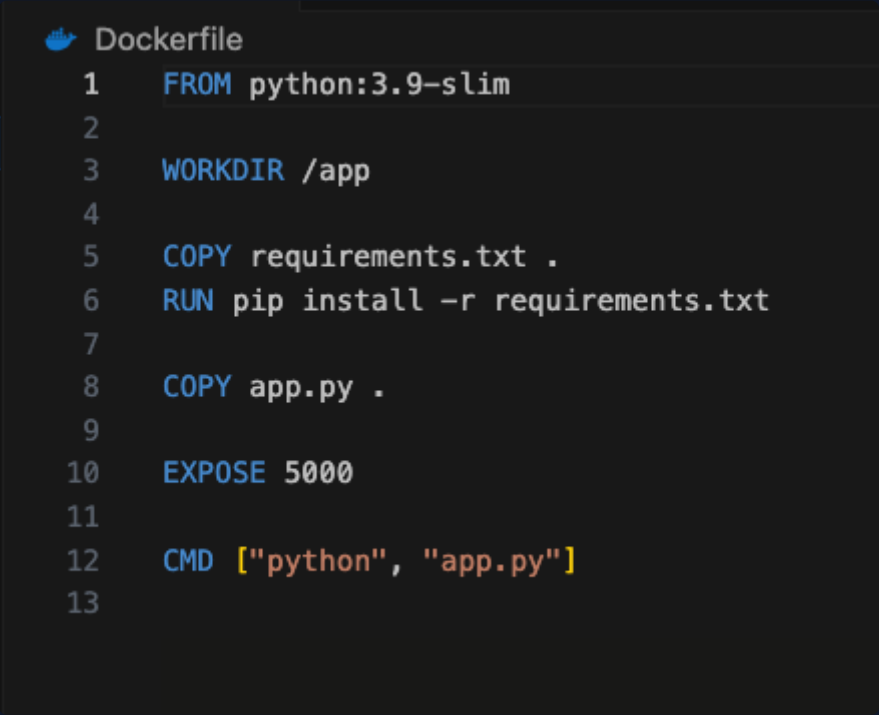
Python Flask application code, which runs a simple web server that returns the message "Hello from my Dockerized app!".

A screenshot of a code editor with a dark background. The file name 'app.py' is shown at the top left with a Python icon. The code is as follows:

```
1  from flask import Flask
2  app = Flask(__name__)
3
4  @app.route('/')
5  def home():
6      return "Hello from my Dockerized app!"
7
8  if __name__ == '__main__':
9      app.run(host='0.0.0.0', port=5000)
10
```

2. Dockerfile

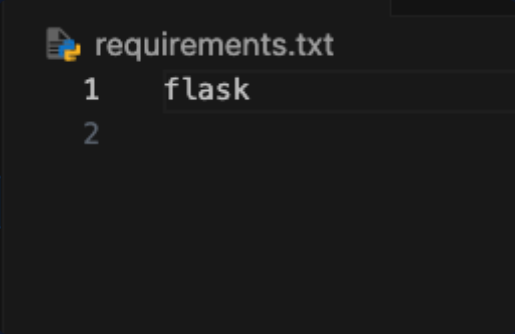
Dockerfile with the base image (python:3.9-slim), working directory setup, dependency installation, and instructions to run the app.

A screenshot of a code editor showing a Dockerfile. The file is titled 'Dockerfile' with a blue icon. The content consists of 13 lines of Docker instructions, numbered 1 through 13 on the left margin. The instructions are: 1. FROM python:3.9-slim, 2. (blank), 3. WORKDIR /app, 4. (blank), 5. COPY requirements.txt ., 6. RUN pip install -r requirements.txt, 7. (blank), 8. COPY app.py ., 9. (blank), 10. EXPOSE 5000, 11. (blank), 12. CMD ["python", "app.py"], 13. (blank).

```
Dockerfile
1  FROM python:3.9-slim
2
3  WORKDIR /app
4
5  COPY requirements.txt .
6  RUN pip install -r requirements.txt
7
8  COPY app.py .
9
10 EXPOSE 5000
11
12 CMD ["python", "app.py"]
13
```

3. requirements.txt

Requirements file listing the Flask dependency required for the application.

A screenshot of a code editor showing a requirements.txt file. The file is titled 'requirements.txt' with a blue icon. The content consists of two lines, numbered 1 and 2 on the left margin. Line 1 contains the word 'flask'. Line 2 is blank.

```
requirements.txt
1  flask
2
```

Part B — Docker Execution Steps

4. Docker Image Build

Running the `docker build -t my-flask-app .` command to successfully build the Docker image.

```
● samirhusayn@Samirs-MacBook-Pro Docker_Task % docker build -t my-flask-app .
[+] Building 24.9s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 189B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 3.2s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f75 8.4s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f75 0.0s
=> => sha256:ea56f685404adf81680322f152d2cfec62115b30dda481c2c450078315beb508 251B / 251B 0.4s
=> => sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c0 13.88MB / 13.88MB 1.3s
=> => sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d 1.29MB / 1.29MB 1.7s
=> => sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5 29.78MB / 29.78MB 6.0s
=> => extracting sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5d 1.3s
=> => extracting sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d 0.1s
=> => extracting sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c08 0.8s
=> => extracting sha256:ea56f685404adf81680322f152d2cfec62115b30dda481c2c450078315beb508 0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 271B                                       0.0s
=> [2/5] WORKDIR /app                                           0.3s
=> [3/5] COPY requirements.txt .                                0.0s
=> [4/5] RUN pip install -r requirements.txt                    11.4s
=> [5/5] COPY app.py .                                          0.1s
=> exporting to image                                           1.2s
=> => exporting layers                                              0.7s
=> => exporting manifest sha256:87a0a61fcd7cc3cda82d65eca8627b47c9dff4e9c3b4cba1e4c1c304769800 0.0s
=> => exporting config sha256:44681601374861c2be25f75c0d69b80c26b6f2930207520a21f7acbacda0dff6 0.0s
=> => exporting attestation manifest sha256:687164daafd20a4e4133da026d88dd4d07f9046a5ec039b354 0.0s
=> => exporting manifest list sha256:7a34cc7c602ab5ebf2bfda62a582bc0cf465edabb7e2ff4c872bce00c 0.0s
=> => naming to docker.io/library/my-flask-app:latest              0.0s
=> => unpacking to docker.io/library/my-flask-app:latest          0.4s
```

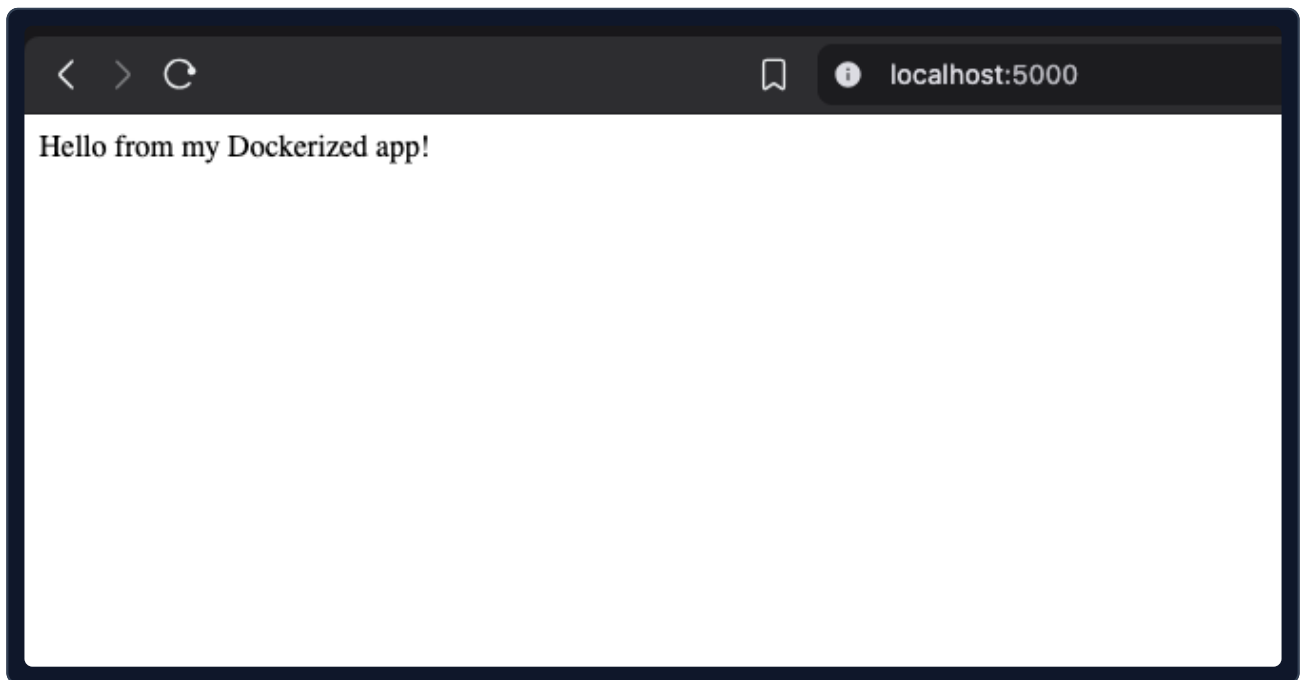
5. Running the Container

Executing `docker run -d -p 5000:5000 my-flask-app` command to start the container in detached mode and map port 5000.

```
samirhusayn@Samirs-MacBook-Pro Docker_Task % docker run -d -p 5000:5000 my-flask-app
dd93b578140b896325831b6793deb7ae961f040f1be4858647526d0b7e95eadf
```

6. Application Output in Browser

The application running at `localhost:5000` in the browser, displaying "Hello from my Dockerized app!".



7. Checking Running Containers

Output of `docker ps` command, displaying the container ID, status, and mapped ports.

```
● samirhusayn@Samirs-MacBook-Pro Docker_Task % docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
dd93b578140b   my-flask-app   "python app.py"         39 seconds ago Up 39 seconds  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   gracious_tharp
```

8. Pulling Image from Docker Hub

Running the `docker pull nginx` command to download the official Nginx image from Docker Hub.

```
● samirhusayn@Samirs-MacBook-Pro Docker_Task % docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
f340c1b7c1d6: Pull complete
a44b5c8be616: Pull complete
02fc02c4ab8d: Pull complete
6310eb16bf42: Pull complete
956faab5efb3: Pull complete
07db7bf2649b: Pull complete
c12f394dea35: Pull complete
76f27c02d218: Download complete
25202a7045eb: Download complete
Digest: sha256:05b8cb60c354a44ab824ea6e7dc69b46d50762cdbe728a347a5b656e6fb3d7c4
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

9. Listing Docker Images

Output of the `docker images` command, listing both local images (`my-flask-app` and `nginx`).

```
docker.io/library/nginx:latest
samirhusayn@Samirs-MacBook-Pro Docker_Task % docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
my-flask-app:latest	7a34cc7c602a	203MB	49.7MB	 Info →  In Use
nginx:latest	05b8cb60c354	253MB	69.2MB	

```
○ samirhusayn@Samirs-MacBook-Pro Docker_Task %
```

Conclusion

This practical lab task provided hands-on experience with fundamental Docker concepts and workflows. Throughout the exercise, key topics were covered including defining application environments with a custom Dockerfile, installing dependencies, building container images, deploying containers locally with port mapping, and pulling/managing images from Docker Hub.